



DESARROLLO DE SOFTWARE PARA PLATAFORMAS MOVILES

JONATHAN LÓPEZ LONDOÑO
JLOPEZL@UAO.EDU.CO
315 926 5443



Ionic - React routing

IonReactRouter

The main routing container in Ionic React.

Internally, it uses React Router's BrowserRouter, but adapted so that Ionic can handle:

- ✓ Mobile app-style animations
- ✓ Native-style history
- ✓ Gesture-based back navigation
- ✓ Screen transitions

```
// App.tsx
const App: React.FC = () => (
  <IonApp>
    <IonReactRouter> ...
  </IonReactRouter>
</IonApp>
);
```

Ionic - React routing

IonReactRouter

The main routing container in Ionic React.

Internally, it uses React Router's BrowserRouter, but adapted so that Ionic can handle:

- ✓ Mobile app-style animations
- ✓ Native-style history
- ✓ Gesture-based back navigation
- ✓ Screen transitions

```
// App.tsx
const App: React.FC = () => (
  <IonApp>
    <IonReactRouter>
      <IonRouterOutlet> ...
    </IonRouterOutlet>
  </IonReactRouter>
</IonApp>
);
```

Ionic - React routing

Route

This is from React Router.

- ✓ It defines which component is rendered based on the URL.

```
// App.tsx
const App: React.FC = () => (
  <IonApp>
    <IonReactRouter>
      <IonRouterOutlet>
        <Route exact path="/home">
          <Home />
        </Route>
        <Route exact path="/">
          <Redirect to="/home" />
        </Route>
      </IonRouterOutlet>
    </IonReactRouter>
  </IonApp>
);
```

Ionic - React routing

routerLink

A feature of Ionic. It allows to navigate without manually using useHistory().

Example:

```
<IonButton routerLink="/home">
```

Go to Home

```
</IonButton>
```

```
// App.tsx
const App: React.FC = () => (
  <IonApp>
    <IonReactRouter>
      <IonRouterOutlet>
        <Route exact path="/home">
          <Home />
        </Route>
        <Route exact path="/">
          <Redirect to="/home" />
        </Route>
      </IonRouterOutlet>
    </IonReactRouter>
  </IonApp>
);
```

Ionic - React routing

useHistory

We can redirect in logic, by using the hook useHistory:

```
import { useIonRouter } from "@ionic/react";
```

```
const history = useHistory();
```

```
const redirect = () => {  
  history.push("/home")  
}
```

```
<IonButton onClick={redirect}>
```

Go to Home

```
</IonButton>
```

```
import { IonButton, IonContent, IonItem, IonLabel, IonList, IonPage } from '@ionic/react'  
import { useHistory } from 'react-router'  
  
export const List = () => {  
  
  const history = useHistory()  
  const redirect = () => {  
    history.push('/detail')  
  }  
  
  return (  
    <IonPage>  
      <IonContent>  
        <IonList>  
          <IonButton  
            onClick={redirect}>  
          >  
            <IonLabel>Item 1</IonLabel>  
          </IonButton>  
        </IonList>  
      </IonContent>  
    </IonPage>  
  )  
}
```

Dynamic routing

To create a dynamic route, the route should be defined like this:

```
<Route path="/detail/:id" exact>  
  <Detail />  
</Route>
```

:id is a dynamic parameter

Then, the route can be used as follow:

```
<IonItem routerLink={'/detail/10'}>  
<IonItem routerLink={`/detail/${contact.id}`}>
```

```
// App.tsx  
const App: React.FC = () => (  
  <IonApp>  
    <IonReactRouter>  
      <IonRouterOutlet>  
        <Route exact path="/home">  
          <Home />  
        </Route>  
        <Route exact path="/detail/:id">  
          <Detail />  
        </Route>  
        <Route exact path="/">  
          <Redirect to="/home" />  
        </Route>  
      </IonRouterOutlet>  
    </IonReactRouter>  
  </IonApp>  
)
```

Dynamic routing

To retrieve the dynamic parameter, we should use `useParams`:

```
import { useParams } from 'react-router'
```

...

```
const { id } = useParams<{ id: string }>();
```

```
console.log(id);
```

```
import { useParams } from 'react-router'

export const Detail = () => {
  const { id } = useParams<{ id: string }>()
  console.log(id)

  return (
    <div>Detail {id}</div>
  )
}
```

State Navigation

Also, we can send bigger data through pages using state navigation:

Example, let's suppose we have an object:

```
const contact = { id: 10, name: "Juan", phone: "3001234567" }
```

Instead of using dynamic routes:

```
<IonItem routerLink={` /detail/${contact.id} }>
```

We can use state navigation as follow:

✓ Define the route as static:

```
<Route path="/detail" exact>
```

✓ Send data through state:

```
<IonItem routerLink="/detail" routerDirection="forward" state={{ contact }}>
```

```
export const List = () => {  
  
  const contact = {  
    id: 10,  
    name: 'Juan',  
    phone: '123456789'  
  }  
  
  return (  
    <IonPage>  
      <IonContent>  
        <IonList>  
          <IonItem  
            routerLink="/detail"  
            state={{ contact }}  
            routerDirection="forward"  
          >  
            <IonLabel>Item 1</IonLabel>  
          </IonItem>  
        </IonList>  
      </IonContent>  
    </IonPage>  
  )  
}
```

Dynamic routing

To retrieve the state parameter, we should use `useLocation`:

```
import {useLocation } from 'react-router'
```

```
...
```

```
const location = useLocation<any>()
```

```
const contact = location.state?.contact
```

```
import { useLocation } from 'react-router-dom'

export const Detail = () => {
  const location = useLocation<any>()
  const contact = location.state?.contact

  console.log(contact)

  return (
    <div>Detail {contact?.name ?? 'No Contact Data'}</div>
  )
}
```

Practice 02

Based on the React app from the Practice 01:

- Create 2 new pages to create and show details from the contacts.
- Use new components from ionic to enhance the app.



Persistence - Storage

localStorage

A browser API that allows data to be saved persistently on the user's device.

A little key – value DataBase in the browser

To store:

```
localStorage.setItem("token", "123456");
```

Objects and arrays must be transformed to string, using `JSON.stringify(...)`

To retrieve:

```
const token = localStorage.getItem("token");
```

Objects and arrays must be transformed from string, using `JSON.parse(...)`

To remove anything

```
localStorage.removeItem("token");
```

To clear storage:

```
localStorage.clear();
```

```
export const List = () => {  
  
  const contact = {  
    id: 10,  
    name: 'Juan',  
    phone: '123456789'  
  }  
  
  localStorage.setItem('contact', JSON.stringify(contact))  
  const contact2 = JSON.parse(localStorage.getItem('contact') || '{}')  
  
  return (  
    <IonPage>  
      <IonContent>  
        <IonList>  
          <IonItem  
            routerLink="/detail"  
            state={{ contact }}  
            routerDirection="forward"  
          >  
            <IonLabel>Item 1</IonLabel>  
          </IonItem>  
        </IonList>  
      </IonContent>  
    </IonPage>  
  )  
}
```

Challenge 04

Create a new Demo Login Page in ionic.
It should contain:

- ✓ Email
- ✓ Password
- ✓ Button to Login
- ✓ When the button is clicked, we are going to validate if the user is:

user@mail.com

password: 123

In that case, we are going to store a token called logged = true and redirect to List page.

- ✓ Next time we enter to the app, we're going to check if the user is logged, then is not necessary to log-in again.
- ✓ Create a button to logout, then when the button is clicked, the app will clean the token and redirects to login page

